

ECS Fargate

Task Definitions · Auto Scaling · Blue/Green · ECR · VPC Design · Cloud Map

AWS Tutorial · ShopFlow

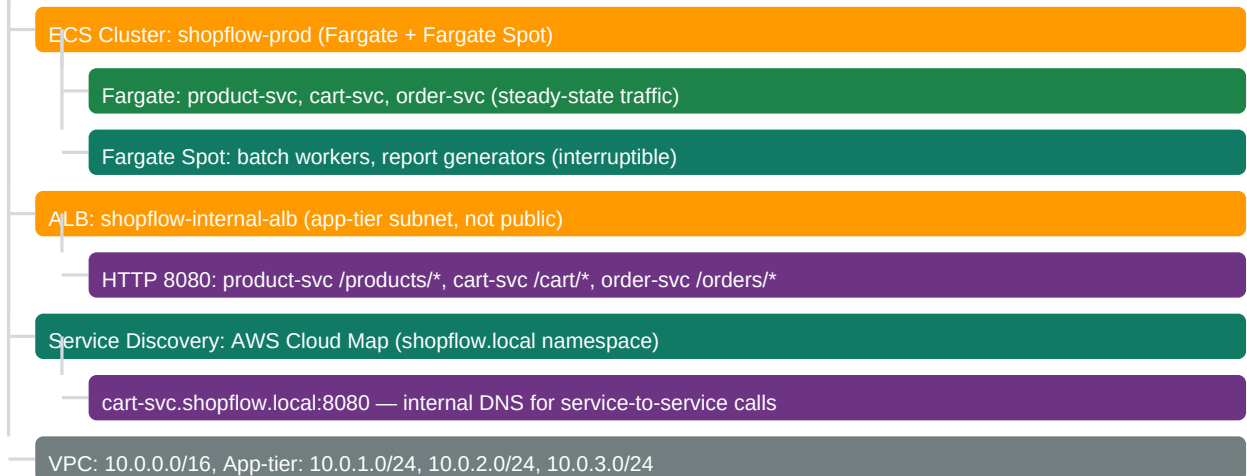
11-00

Chapter Overview — ECS Fargate at ShopFlow

ShopFlow runs all synchronous microservices on Amazon ECS Fargate: containerized Java 21 + Spring Boot 3 services with no EC2 instance management. Fargate handles OS patching, host provisioning, and bin-packing. Services scale independently based on CPU, memory, and custom metrics via Application Auto Scaling.

ECS Service	Container Spec	Replicas (Normal/BF)	Trigger for Scale-Out
product-svc	1 vCPU / 2 GB, Java 21	3 / 15	CPU > 60% for 3 min
cart-svc	0.5 vCPU / 1 GB, Java 21	3 / 10	ALB RequestCount > 500/target
order-svc	1 vCPU / 2 GB, Java 21	3 / 12	CPU > 70% for 2 min
user-svc	0.5 vCPU / 1 GB, Java 21	2 / 5	CPU > 70%
search-svc	1 vCPU / 2 GB, Java 21	3 / 10	CPU > 60%
notification-svc	0.25 vCPU / 0.5 GB, Java 21	2 / 4	SQS depth > 500

ShopFlow ECS Cluster Architecture



■ Fargate Spot uses spare EC2 capacity at up to 70% discount. Tasks can be interrupted with 2-minute warning. Use Fargate Spot for: batch report generators, image resizers, background indexers, dev/test environments. NEVER use Spot for stateful services or anything requiring <2-minute graceful shutdown. ShopFlow Spot tasks handle SIGTERM and flush state to SQS before shutdown.

11-01

ECS Task Definition — Java 21 Spring Boot Container

Task definition specifies container image, CPU/memory, environment variables, IAM task role, log configuration, and health check. ShopFlow uses CDK to generate task definitions — one per service. Secrets are injected via ECS Secrets Manager integration (never in environment variables plaintext).

Task Definition CDK — product-svc

```

const productTaskDef = new ecs.FargateTaskDefinition(this, "ProductTaskDef", {
  family: "shopflow-product-svc",
  cpu: 1024, // 1 vCPU
  memoryLimitMiB: 2048, // 2 GB
  taskRole: productTaskRole, // permissions for DB, Redis, S3
  executionRole: ecsExecutionRole, // pull ECR image, write logs
  runtimePlatform: {
    cpuArchitecture: ecs.CpuArchitecture.ARM64, // Graviton3 Fargate
    operatingSystemFamily: ecs.OperatingSystemFamily.LINUX,
  },
});
productTaskDef.addContainer("product-svc", {
  image: ecs.ContainerImage.fromEcrRepository(ecrRepo, "product-svc-v2.1.0"),
  portMappings: [{ containerPort: 8080, protocol: ecs.Protocol.TCP }],
  environment: { // non-sensitive config only
    SPRING_PROFILES_ACTIVE: "prod",
    AWS_REGION: "us-east-1",
    JAVA_TOOL_OPTIONS: "-XX:+UseG1GC -XX:MaxRAMPercentage=75.0 -Xss256k",
  },
  secrets: { // injected from Secrets Manager
    DB_URL: ecs.Secret.fromSecretsManager(auroraSecret, "host"),
    DB_PASSWORD: ecs.Secret.fromSecretsManager(auroraSecret, "password"),
    REDIS_AUTH_TOKEN: ecs.Secret.fromSecretsManager(redisSecret),
  },
  logging: ecs.LogDrivers.awsLogs({
    streamPrefix: "product-svc",
    logGroup: new logs.LogGroup(this, "ProductLogs", {
      retention: logs.RetentionDays.ONE_MONTH,
    }),
  }),
  healthCheck: {
    command: ["CMD-SHELL", "curl -f http://localhost:8080/actuator/health || exit 1"],
    interval: Duration.seconds(15),
    timeout: Duration.seconds(5),
    retries: 3,
  },
});

```

■ Set `-XX:MaxRAMPercentage=75.0` to cap JVM heap at 75% of container memory. Without this, the JVM defaults to 25% of system RAM — in a 2GB container: only 512MB heap. With `MaxRAMPercentage=75`: 1.5GB heap, leaving 512MB for OS, thread stacks, and off-heap (NIO buffers, metaspace). Never set `-Xmx` directly in ECS — the percentage adapts when you resize the task definition.

11-02

ECS Service — Rolling Deploy, Circuit Breaker, ALB

ECS Service maintains the desired number of running tasks, integrates with ALB for traffic routing, and performs rolling deployments. The deployment circuit breaker automatically rolls back if the new task version fails health checks — preventing prolonged outages from bad deployments.

ECS Service CDK

```

const productService = new ecs.FargateService(this, "ProductSvc", {
  cluster,
  taskDefinition: productTaskDef,
  desiredCount: 3,
  serviceName: "shopflow-product-svc",
  vpcSubnets: {
    subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS,
  },
  securityGroups: [sgProductSvc],
  deploymentController: {
    type: ecs.DeploymentControllerType.ECS, // rolling update (not Blue/Green)
  },
});

```

```

},
circuitBreaker: {
  rollback: true, // auto-rollback on deployment failure
},
deploymentConfiguration: {
  maximumPercent: 200, // allow 2x tasks during rolling deploy
  minimumHealthyPercent: 50, // keep at least 50% healthy during deploy
},
enableExecuteCommand: true, // ECS Exec for live debugging (SSM-based)
cloudMapOptions: {
  name: "product-svc",
  cloudMapNamespace: shopflowNamespace, // product-svc.shopflow.local
},
platformVersion: ecs.FargatePlatformVersion.LATEST,
capacityProviderStrategies: [
  { capacityProvider:"FARGATE", weight:1, base:3 },
  { capacityProvider:"FARGATE_SPOT", weight:0 }, // Spot for burst
],
});
// Register with ALB target group
productService.attachToApplicationTargetGroup(productTargetGroup);

```

■ Deployment circuit breaker monitors the health check of newly launched tasks. If >50% of new tasks fail their health check in the deployment window (10 minutes default), ECS triggers automatic rollback to the previous task definition. This prevents a bad deployment from bringing all tasks down. ALWAYS enable `rollback: true` in production ECS services — it is off by default.

11-03

Application Auto Scaling — CPU & Request-Based

ECS services scale via Application Auto Scaling. ShopFlow uses two scaling policies: Target Tracking on CPU (scale to maintain 60% average CPU) and Step Scaling on ALB RequestsPerTarget (scale aggressively when request rate spikes). Both policies work simultaneously — the most aggressive wins.

Auto Scaling CDK

```

const scaling = productService.autoScaleTaskCount({
  minCapacity: 3,
  maxCapacity: 50, // Black Friday: allow up to 50 tasks
});
// Policy 1: Target Tracking – maintain 60% average CPU
scaling.scaleOnCpuUtilization("CpuScaling",{
  targetUtilizationPercent: 60,
  scaleInCooldown: Duration.minutes(5), // wait 5min before scaling in
  scaleOutCooldown: Duration.seconds(60), // scale out fast – 60s cooldown
});
// Policy 2: Target Tracking – ALB requests per target
scaling.scaleOnRequestCount("RequestScaling",{
  requestsPerTarget: 500, // scale when avg task sees > 500 req/min
  targetGroup: productTargetGroup,
  scaleInCooldown: Duration.minutes(5),
  scaleOutCooldown: Duration.seconds(60),
});
// Policy 3: Scheduled scale-out for Black Friday (predictive)
scaling.scaleOnSchedule("BlackFridayScaleOut",{
  schedule: appscaling.Schedule.expression("cron(0 6 29 11 ? *)"),
  minCapacity: 15, // pre-scale to 15 tasks at 6am on Black Friday
  maxCapacity: 50,
});

```

Scaling Metric	ScaleOut Trigger	ScaleIn Trigger	Cooldown (out/in)	Best For
CPU Target Tracking	Avg CPU > 60%	Avg CPU < 60%	60s / 5min	General-purpose compute scaling

Scaling Metric	ScaleOut Trigger	ScaleIn Trigger	Cooldown (out/in)	Best For
Request Count Target	Req/target > 500/min	Req/target < 500/min	60s / 5min	HTTP API services behind ALB
Memory Target Tracking	Avg MEM > 75%	Avg MEM < 75%	60s / 5min	Memory-heavy cache or ML services
Step Scaling (SQS depth)	Queue > 500 msgs	Queue < 100 msgs	30s / 3min	Worker services consuming SQS qu

■ Target Tracking scale-in cooldown should be 3-5 minutes (not the same as scale-out). Rapid scale-in after a traffic spike causes thrashing: scale out 3→10 tasks, then immediately scale in 10→3, then scale out again. Set scaleInCooldown=5min to let the service absorb traffic variation before reducing capacity. Scale-out should be aggressive (60s); scale-in should be conservative (5min).

11-04

Fargate Spot — Cost Optimization for Batch Workloads

Fargate Spot uses spare AWS capacity at up to 70% discount. Spot tasks receive a 2-minute SIGTERM warning before interruption. ShopFlow runs image resizing, catalog exports, and report generation on Spot — these tasks checkpoint progress to SQS/S3 and resume cleanly after interruption.

Workload	Fargate (On-Demand)	Fargate Spot	Monthly Savings
product-svc (3 tasks × 1vCPU/2GB)	\$0.04048/hr × 730hr = \$88.7	Not used — stateful, cannot interrupt	\$0
image-resizer (2 tasks × 0.5vCPU/1GB)	\$0.02024/hr × 730hr = \$29.6	2 × \$0.00607/hr × 730hr = \$8.92	\$20.7 (70% off)
catalog-exporter (1 task × 2vCPU/4GB)	\$0.02996/hr × 730hr = \$59.1	\$0.02429/hr × 730hr = \$17.7	\$41.4 (70% off)
daily-report (1 task × 1vCPU/2GB)	\$0.01408/hr × 30hr = \$1.21	\$0.01214/hr × 30hr = \$0.36	\$0.85

Graceful Spot Interruption Handler — Spring Boot

```
// ECS sends SIGTERM 2 minutes before Spot interruption
@Component
public class SpotInterruptionHandler {
    @EventListener
    public void onContextClosed(ContextClosedEvent event) {
        log.warn("SIGTERM received – Spot interruption imminent. Flushing state.");
        // 1. Stop accepting new work items from SQS
        sqsListenerContainer.stop();
        // 2. Complete in-flight items (up to 90 seconds max)
        long deadline = System.currentTimeMillis() + 90_000;
        while (hasInFlightWork() && System.currentTimeMillis() < deadline) {
            Thread.sleep(1000);
        }
        // 3. Re-enqueue incomplete work back to SQS for next instance
        incompleteWork.forEach(item -> sqsClient.sendMessage(item.toSqsRequest()));
        log.warn("Graceful shutdown complete. {} items re-queued.", incompleteWork.size());
    }
}
```

■ Capacity Providers let you mix Fargate and Fargate Spot within one ECS service using weights and base values. base=3 FARGATE ensures at least 3 on-demand tasks always run (service availability guarantee). Additional tasks beyond 3 use the weight ratio: weight=1 FARGATE + weight=3 FARGATE_SPOT means 75% of additional tasks use Spot. Spot saves 70% on the incremental capacity during Black Friday bursts.

11-05

ECR — Container Image Management & Lifecycle

Amazon ECR stores ShopFlow Docker images. Each service has a dedicated ECR repository with lifecycle policies to retain only the last 10 tagged images (preventing storage bloat). Image scanning on push detects CVEs. Cross-region replication copies images to ap-southeast-1 for DR.

ECR Repository CDK

```
const productEcr = new ecr.Repository(this, "ProductSvcRepo", {
  repositoryName: "shopflow/product-svc",
  imageScanOnPush: true, // automatic CVE scanning on docker push
  encryption: ecr.RepositoryEncryption.KMS,
  imageTagMutability: ecr.TagMutability.IMMUTABLE, // tags cannot be overwritten
  lifecycleRules: [
    // Keep last 10 tagged releases
    { rulePriority: 1, description: "Keep last 10 releases",
      tagStatus: ecr.TagStatus.TAGGED,
      tagPatternList: ["v*"], // only tags matching "v*"
      maxImageCount: 10 },
    // Delete untagged images older than 7 days (from CI build cache)
    { rulePriority: 2, description: "Delete old untagged",
      tagStatus: ecr.TagStatus.UNTAGGED,
      maxImageAge: Duration.days(7) },
  ],
});

// Cross-region replication for DR
new ecr.CfnReplicationConfiguration(this, "EcrReplication", {
  replicationConfiguration: {
    rules: [{
      destinations: [{ region: "ap-southeast-1", registryId: accountId }],
      repositoryFilters: [{ filter: "shopflow/",
        filterType: "PREFIX_MATCH" }],
    }],
  },
});
```

ECR Feature	Config	Benefit
Image scanning (Enhanced)	Amazon Inspector + ECR Scan	CVE detection on push; severity: CRITICAL blocks CI/CD pipeline
Immutable tags	imageTagMutability: IMMUTABLE	v2.1.0 always points to same image; no tag overwrite accidents
Lifecycle policy	Keep last 10 tagged; delete untagged >7d	Prevent ECR storage bloat (\$0.10/GB/month; 100GB = \$10/mo)
Pull-through cache	ecr.createPullThroughCacheRule (Docker Hub, ECR Public)	Cache public base images (amazoncorretto:21-al2023) in private ECR
Cross-region replication	Replicate shopflow/* to ap-southeast-1	DR: images available for failover without cross-region pull latency

■ Use image tag immutability in production. With mutable tags, docker push v2.1.0 can silently overwrite a previously deployed image — ECS rolling deploy could load a different binary than expected. With IMMUTABLE: push v2.1.0 twice → ECR error. This forces a version bump for every deployment and guarantees deployed image = tested image.

11-06

ECS Exec — Live Container Debugging

ECS Exec enables an interactive shell into a running Fargate container without SSH or a bastion host. It uses AWS SSM Session Manager as the transport — no inbound ports required. ShopFlow enables ECS Exec on all services for incident response, disabled via IAM in production by default.

ECS Exec — Debugging a Running Task

```
# List running tasks for product-svc
aws ecs list-tasks --cluster shopflow-prod --service-name shopflow-product-svc
# Output: task ARNs, e.g. arn:aws:ecs:us-east-1:...:task/shopflow-prod/abc123
# Open interactive shell into the running container
aws ecs execute-command \
  --cluster shopflow-prod \
  --task abc123def456 \
  --container product-svc \
  --interactive \
  --command "/bin/bash"
# Once inside the container:
```

```
# Check JVM heap usage
jcmd 1 VM.native_memory summary
# Trigger heap dump to /tmp (then copy to S3)
jmap -dump:format=b,file=/tmp/heap.hprof 1
aws s3 cp /tmp/heap.hprof s3://shopflow-debug/heap-$(date +%Y%m%d).hprof
# Check thread state – identify deadlocks
jstack 1 | grep -A 5 "BLOCKED"
# Test Redis connectivity from inside the container
nc -zv ${REDIS_ENDPOINT} 6379 && echo "Redis OK"
```

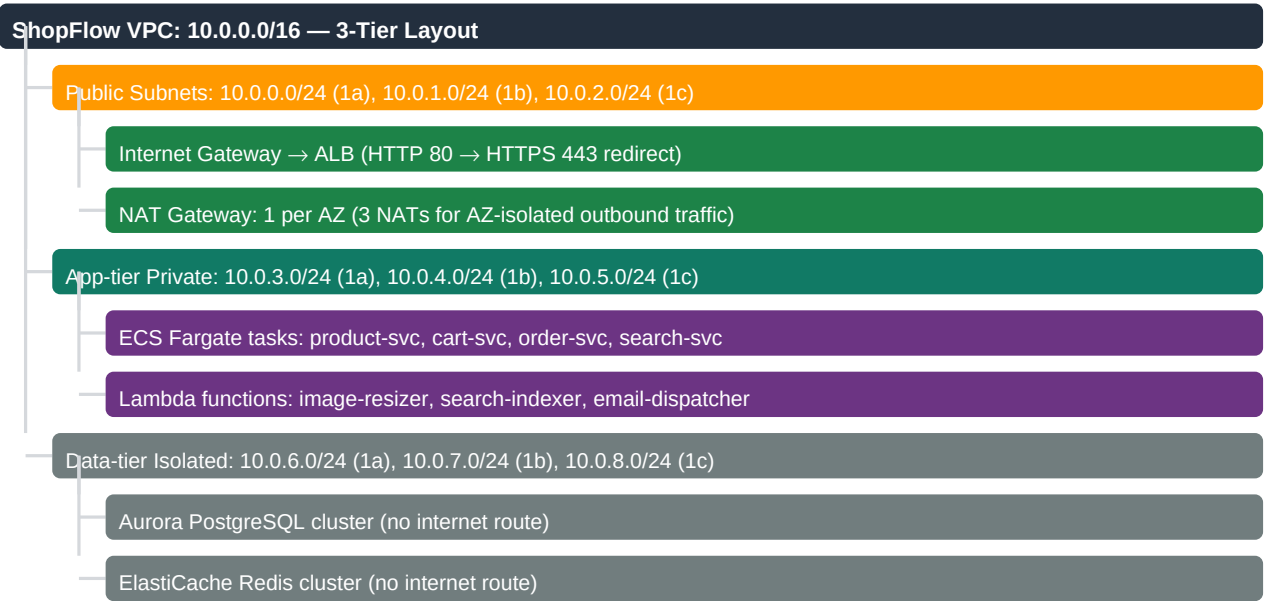
ECS Exec Requirement	Config	Notes
SSM Agent in container	Amazon Linux 2023 base image includes SSM Agent	Custom base images: install amazon-ssm-agent
Task role IAM	ssm:StartSession, ssm:TerminateSession, ssm:SendCommand	Add to task role — NOT execution role
enableExecuteCommand	enableExecuteCommand: true in FargateService CDK	Must be set before task launch — cannot enable after
VPC: SSM endpoints	Interface endpoint: com.amazonaws.us-east-1.ssm	Required if ECS tasks have no internet access

■ ECS Exec grants shell access to the running production container. Restrict via IAM: production engineers should require MFA and time-limited assume-role to access the ECS Exec IAM permission (ssm:StartSession on resource ecs:task/*). Log all ECS Exec sessions to CloudTrail. Never leave ECS Exec enabled in production without IAM guardrails — it bypasses all network security controls.

11-07

VPC Network Design — 3-Tier Security

ShopFlow VPC uses a 3-tier design: public subnets (ALB, NAT Gateway), app-tier private subnets (ECS tasks, Lambda), and data-tier isolated subnets (Aurora, Redis, DynamoDB VPC endpoints). Each tier has separate Security Groups with minimal inbound rules.



Security Group Rules — Product-svc to Aurora

```
// sg-alb: receives traffic from internet (0.0.0.0/0 -> 443)
sgAlb.addIngressRule(ec2.Peer.anyIpv4(), ec2.Port.tcp(443));
// sg-product-svc: receives traffic only from ALB SG
sgProductSvc.addIngressRule(sgAlb, ec2.Port.tcp(8080),
"ALB -> product-svc container port");
// sg-aurora: receives traffic only from ECS task SG
sgAurora.addIngressRule(sgProductSvc, ec2.Port.tcp(5432),
"product-svc -> Aurora PostgreSQL");
```

```
sgAurora.addIngressRule(sgOrderSvc, ec2.Port.tcp(5432),
"order-svc → Aurora PostgreSQL");
// No public ingress to Aurora – not reachable from internet
```

Tier	Subnet Type	Internet Access	SG Rules
Public (ALB, NAT)	PUBLIC (IGW route)	Inbound + outbound internet	ALB: 443 from 0.0.0.0/0
App (ECS, Lambda)	PRIVATE_WITH_EGRESS (NAT route)	Outbound only via NAT	ECS: 8080 from ALB SG only
Data (Aurora, Redis)	PRIVATE_ISOLATED (no NAT)	None — VPC endpoints only	Aurora: 5432 from ECS SG only; Redis: 6379 from ECS SG only

11-08

Service Discovery — AWS Cloud Map

AWS Cloud Map provides DNS-based service discovery within the VPC. ECS registers each task's private IP at launch. Callers use DNS name (cart-svc.shopflow.local) instead of hardcoded IPs. Health checks remove unhealthy tasks from DNS automatically. ShopFlow uses Cloud Map for all internal service-to-service calls.

Cloud Map CDK Setup

```
// Private DNS namespace – resolves only within VPC
const shopflowNamespace = new servicediscovery.PrivateDnsNamespace(
this, "ShopFlowNamespace", {
name: "shopflow.local",
vpc,
description: "ShopFlow internal service discovery",
});
// ECS Service: register each task IP in Cloud Map
cartService = new ecs.FargateService(this, "CartSvc", {
...
cloudMapOptions: {
name: "cart-svc", // cart-svc.shopflow.local
cloudMapNamespace: shopflowNamespace,
dnsRecordType: servicediscovery.DnsRecordType.A, // IP records
dnsTtl: Duration.seconds(10), // fast failover on task restart
},
});
```

Service-to-Service Call — order-svc calls cart-svc

```
// Spring RestClient: DNS resolves to healthy task IPs (Round-robin)
@Configuration
public class ServiceConfig {
@Bean
public RestClient cartClient() {
return RestClient.builder()
.baseUrl("http://cart-svc.shopflow.local:8080")
.defaultHeader("Content-Type", "application/json")
.build();
}
}
// Call: cartClient.get().uri("/cart/{userId}").retrieve().body(CartDto.class)
// DNS: cart-svc.shopflow.local → [10.0.4.12, 10.0.4.23, 10.0.5.41]
// Spring picks one IP per request (round-robin at DNS client)
```

Discovery Method	ShopFlow Use	Latency	Health Check Propagation
AWS Cloud Map (DNS)	Internal service-to-service (order-svc → cart-svc)	DNS TTL 10s propagation	Unhealthy task removed from DNS within TTL+5s
ALB Target Group	External (API GW → ALB → ECS)	Real-time deregistration	ALB health check: 15s interval; instant removal on fail
Hardcoded IP	Never in production	Fast — no DNS	Manual update required on task restart

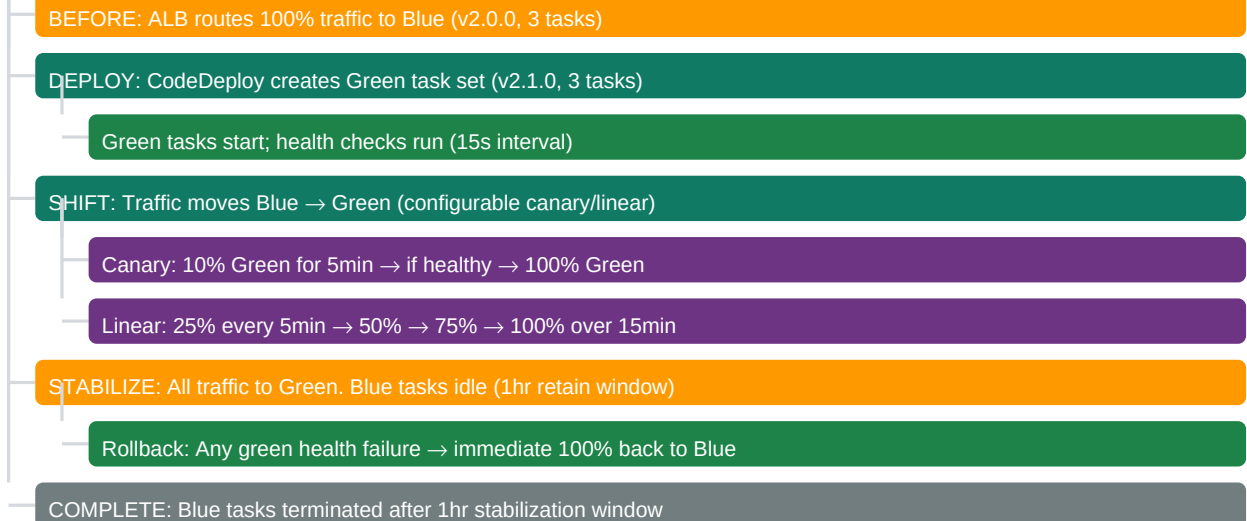
■ Set Cloud Map DNS TTL to 10-15 seconds. The default 60-second TTL means a dead task's IP stays in DNS for up to 60 seconds after removal — callers will get connection refused for 60 seconds. At 10s TTL, the failed task is removed from caller DNS caches within 10s. The trade-off: slightly higher DNS query volume. For 10 services × 1,000 RPS × 10s TTL = 100 DNS queries/s — negligible.

11-09

Blue/Green Deployment with CodeDeploy

Blue/Green deployment creates a new "green" task set alongside the existing "blue" tasks. Traffic shifts from blue to green using ALB weighted target groups. If green fails health checks, CodeDeploy rolls back to blue instantly. Zero-downtime for order-svc deployments where rolling update creates brief instability.

Blue/Green Deployment Flow — order-svc



Blue/Green CDK

```
const orderService = new ecs.FargateService(this, "OrderSvc", {
  ...
  deploymentController: {
    type: ecs.DeploymentControllerType.CODE_DEPLOY, // Blue/Green via CodeDeploy
  },
});
const deploymentGroup = new codedeploy.EcsDeploymentGroup(
  this, "OrderBlueGreen", {
    service: orderService,
    blueGreenDeploymentConfig: {
      blueTargetGroup: blueTargetGroup,
      greenTargetGroup: greenTargetGroup,
      listener: prodListener, // ALB 443 listener
      testListener: testListener, // ALB 8080 test port (optional)
      terminationWaitTime: Duration.hours(1),
    },
    deploymentConfig:
      codedeploy.EcsDeploymentConfig.CANARY_10PERCENT_5MINUTES,
    alarms: [p99LatencyAlarm, errorRateAlarm], // auto-rollback on alarm
  });
```

■ CodeDeploy Blue/Green adds ~5 minutes to deployment time compared to rolling update (task startup + traffic shift + stabilization). Use Blue/Green for order-svc and cart-svc (financial transactions — zero-downtime critical). Use rolling update for product-svc and search-svc (reads — brief extra latency on deploy is acceptable). The cost: Blue/Green requires 2× ECS tasks during the deployment window (~10-20 minutes × 2× cost).

ALB health checks determine if an ECS task receives traffic. Deregistration delay is the time ALB waits after marking a task unhealthy before sending a 503 — allowing in-flight requests to complete. Spring Boot Actuator /health endpoint is the standard health check path.

Config	ShopFlow Value	Effect	Wrong Value Risk
Health check path	/actuator/health	Returns 200 (UP) or 503 (DOWN) based on DB + Redis	200 even if Spring crashed; misleading
Health check interval	15s	Check every 15s; detect failure within 15-45s	60s → 60s delay detecting bad task
Healthy threshold	3 consecutive successes	New task receives traffic after 3 passing checks = 45s	1 → traffic before Spring context fully initialized
Unhealthy threshold	2 consecutive failures	Task marked unhealthy after 2 failures = 30s	5 → bad task serves traffic 75s too long
Deregistration delay	30s	ALB waits 30s before draining connection	300s (default) → rolling deploy takes 5min per task

Spring Boot Actuator Health — Custom Checks

```
// application.yml: expose health endpoint with detailed component status
management:
  endpoint:
    health:
      show-details: always # or when-authorized for security
  group:
    liveness: # separate liveness (crash) vs readiness (traffic)
    include: livenessState
  readiness:
    include: readinessState, db, redis, diskSpace
  endpoints:
    web:
      base-path: /actuator
      exposure:
        include: health, info, metrics, prometheus
// Custom health indicator – check Aurora connection pool
@Component
public class AuroraHealthIndicator implements HealthIndicator {
  @Override
  public Health health() {
    try {
      jdbcTemplate.queryForObject("SELECT 1", Integer.class);
      return Health.up().withDetail("aurora", "connected").build();
    } catch (DataAccessException e) {
      return Health.down(e).withDetail("aurora", "unreachable").build();
    }
  }
}
```

■ ALB deregistration delay default is 300 seconds (5 minutes). For ECS rolling deployments with 3 tasks and desired=3 config, each task deregistration takes 5 minutes → 15+ minutes total for a 3-task rolling deploy. Reduce to 30s for REST APIs (most requests complete in <1s). For WebSocket services, keep at 300s (connection-based — long-lived connections need drain time). Set in ALB Target Group attributes.

CloudWatch Container Insights collects CPU, memory, network, and disk metrics at task and container level. X-Ray traces cross-service requests: API Gateway → product-svc → Aurora/Redis. Structured logging with correlation IDs enables request-level debugging across all services.

Metric	Namespace	Alert Threshold	Action
CpuUtilized	ECS/ContainerInsights	>80% for 5min (task average)	Auto Scaling triggers scale-out (target: 60%)
MemoryUtilized	ECS/ContainerInsights	>85% for 5min	Check for memory leak; take heap dump via ECS Exec
RunningTaskCount	ECS/ContainerInsights	Below minCapacity for 5min	ECS task crashed — check CloudWatch Logs + Errors
NetworkTxBytes	ECS/ContainerInsights	Sustained drop (>50% decrease)	Network isolation or task crash — check SG rules
StorageWriteBytes	ECS/ContainerInsights	Spike >100MB/s per task	Check for log verbosity explosion; audit log level

Container Insights CDK

```
const cluster = new ecs.Cluster(this,"ShopFlowCluster",{
  clusterName: "shopflow-prod",
  vpc,
  containerInsights: true, // enable CloudWatch Container Insights
  enableFargateCapacityProviders: true,
});
// X-Ray sidecar container in each task definition
productTaskDef.addContainer("xray-daemon",{
  image: ecs.ContainerImage.fromRegistry("amazon/aws-xray-daemon"),
  cpu: 32, // 0.032 vCPU – minimal overhead
  memoryReservationMiB: 256,
  logging: ecs.LogDrivers.awsLogs({ streamPrefix: "xray" }),
  portMappings: [{
    containerPort: 2000,
    protocol: ecs.Protocol.UDP,
  }],
});
```

■ X-Ray sampling rules reduce trace volume and cost. Default: 5% of requests traced. For high-value flows (order placement, payment), create a custom sampling rule: sample 100% of requests matching service=order-svc AND urlPath=/orders AND httpMethod=POST. For high-volume product reads: sample 1%. This ensures complete visibility on critical paths without tracing 100% of 10,000 RPS.

11-12 Cost Optimization — Right-Sizing & Savings Plans

Fargate pricing: \$0.04048/vCPU-hr + \$0.004445/GB-hr. Compute Savings Plans give 36-52% discount on committed usage for 1-3 years. Graviton (ARM64) gives 20% cost reduction vs x86 at the same price tier. Right-sizing based on CloudWatch Container Insights reduces waste.

Service	Current Spec	CPU Util Avg	Right-Sized Spec	Monthly Saving
product-svc (3 tasks)	1 vCPU / 2 GB	42%	0.75 vCPU / 1.5 GB	20% — \$17.7/mo
cart-svc (3 tasks)	0.5 vCPU / 1 GB	35%	0.25 vCPU / 0.5 GB	50% — \$8.8/mo
order-svc (3 tasks)	1 vCPU / 2 GB	67%	Keep — near target 60%	\$0 (well-utilized)
search-svc (3 tasks)	1 vCPU / 2 GB	28%	0.5 vCPU / 1 GB	50% — \$26.5/mo

Compute Savings Plan Calculation

```
# ShopFlow steady-state Fargate usage (all services, min replicas):
# product-svc: 3 × (1vCPU × $0.04048 + 2GB × $0.004445) = $0.148/hr
# cart-svc: 3 × (0.5vCPU × $0.04048 + 1GB × $0.004445) = $0.074/hr
# order-svc: 3 × (1vCPU × $0.04048 + 2GB × $0.004445) = $0.148/hr
# user-svc: 2 × (0.5vCPU × $0.04048 + 1GB × $0.004445) = $0.049/hr
# search-svc: 3 × (1vCPU × $0.04048 + 2GB × $0.004445) = $0.148/hr
# Total steady-state: ~$0.567/hr = $413/month
# Compute Savings Plan (1-year, no upfront): ~36% discount
# Commit: $0.567/hr on-demand → $0.363/hr committed
```

```
# Monthly savings: ($0.567 - $0.363) × 720hr = $147/month
# Annual savings: $1,764/year with 1-year commitment
# With Graviton ARM64 (20% discount on Fargate):
# $0.04048/vCPU-hr x86 → $0.03238/vCPU-hr ARM64
# Additional saving on base usage: ~20% of Fargate compute
```

■ Compute Savings Plans apply automatically to any compute usage (Fargate, EC2, Lambda) that matches the commitment. Unlike EC2 Reserved Instances (locked to instance type + region), Compute Savings Plans cover Fargate ARM64, x86, Lambda, and EC2 across all instance families. For ShopFlow mixed Fargate + Lambda: one Compute Savings Plan covers both. Recommend 1-year, no-upfront for maximum flexibility.

11-13

ECS Secrets & Configuration Management

ECS Fargate injects secrets from Secrets Manager and Parameter Store at task launch — values are never in Dockerfile or application code. CDK generates the task definition with secret references. The task execution role (not task role) needs `GetSecretValue` permission to pull secrets during task startup.

Secret / Config	Store	Path	Who Accesses It
Aurora password	Secrets Manager	arn:aws:secretsmanager:.../shopflow/aurora/prod	ECS execution role (task startup); RDS (auto-rotation)
Redis AUTH token	Secrets Manager	arn:aws:secretsmanager:.../shopflow/redis/auth	ECS execution role (startup); app reads REDIS_AUTH
Stripe API key	Secrets Manager	arn:aws:secretsmanager:.../shopflow/stripe/api-key	ecr-dkr task role at runtime via AWS SDK call
Spring profiles	Parameter Store (String)	arn:aws:ssm:.../shopflow/config/spring-profile	ECS task definition: SPRING_PROFILES_ACTIVE env
Log level	Parameter Store (String)	arn:aws:ssm:.../shopflow/config/log-level	ECS task definition: LOG_LEVEL env var

ECS Execution Role — `GetSecretValue`

```
// Execution role needs GetSecretValue to inject secrets at startup
ecsExecutionRole.addToPolicy(new iam.PolicyStatement({
  actions: ["secretsmanager:GetSecretValue"],
  resources: [
    auroraSecret.secretArn,
    redisSecret.secretArn,
    // Do NOT grant all secrets: arn:aws:secretsmanager:*:*:secret:*
    // Grant only specific secrets this service needs
  ],
}));

ecsExecutionRole.addToPolicy(new iam.PolicyStatement({
  actions: ["ssm:GetParameters", "ssm:GetParameter"],
  resources: [
    `arn:aws:ssm:us-east-1:${account}:parameter/shopflow/config/*`,
  ],
}));
```

■ ECS secrets injection happens at task START. Rotating a secret in Secrets Manager does not update running tasks — they hold the secret value from startup until the task is restarted. For password rotation: (1) rotate secret in Secrets Manager, (2) force new ECS deployment (`aws ecs update-service --force-new-deployment`). ECS will launch new tasks with the new secret and drain old tasks. This is the standard rotation procedure for ECS + Secrets Manager.

11-14

ECS Networking — VPC Endpoints & NAT Optimization

ECS Fargate tasks in private subnets need NAT Gateway or VPC endpoints to reach AWS services (ECR, S3, Secrets Manager, CloudWatch). VPC endpoints eliminate NAT costs for high-volume AWS API calls. At 10GB/day ECR pull traffic: NAT cost = \$13.50/month; VPC endpoint = \$0.01/GB = \$3/month.

VPC Endpoint	Type	ShopFlow Services Benefiting	Monthly Saving vs NAT
ecr.api + ecr.dkr	Interface	All ECS services (image pulls on task launch)	ECR pulls: ~5GB/deploy × 10 deploys/mo × \$0.045 =

VPC Endpoint	Type	ShopFlow Services Benefiting	Monthly Saving vs NAT
s3	Gateway (free)	ECS image layers from S3; Lambda code package	Free — Gateway endpoints have no hourly charge
secretsmanager	Interface	ECS execution role GetSecretValue on task start	~\$0.01/task start vs NAT
logs	Interface	CloudWatch Logs PutLogEvents from all ECS tasks	10GB/day logs × 30 × \$0.045 = \$13.5/mo vs \$0.01/GB
ssm + ssmmessages	Interface	ECS Exec (SSM Session Manager transport)	Required for ECS Exec — no NAT path available for SSM

VPC Endpoints CDK

```
// S3 Gateway endpoint – free; all S3 traffic via private backbone
vpc.addGatewayEndpoint("S3Endpoint",{
  service: ec2.GatewayVpcEndpointAwsService.S3,
});
// Interface endpoints – $0.01/hr/AZ + $0.01/GB
["ecr.api","ecr.dkr","secretsmanager","logs","ssm","ssmmessages"]
.forEach(svc => {
  vpc.addInterfaceEndpoint(svc.replace(".", "") + "Endpoint", {
    service: new ec2.InterfaceVpcEndpointService(
      `com.amazonaws.us-east-1.${svc}`),
    subnets: { subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS },
    privateDnsEnabled: true, // reuse existing DNS names (no code change)
    securityGroups: [sgVpcEndpoints],
  });
});
```

■ Interface VPC endpoints cost \$0.01/hour/AZ. For 6 interface endpoints × 3 AZs = 18 endpoint ENIs × \$0.01/hr × 720hr = \$129.60/month. Compare to NAT Gateway: \$0.045/GB × 30GB/day × 30 days = \$40.50/month. NAT is cheaper at low traffic. Break-even: interface endpoints save money when data exceeds 2.88TB/month through the endpoints. For ShopFlow: logs + ECR at ~300GB/month — NAT is cheaper; use Gateway endpoint (free) for S3 only.

11-15 ECS vs EKS — When to Use Kubernetes at ShopFlow

ShopFlow uses ECS Fargate — not Kubernetes (EKS). ECS is simpler for straightforward microservice deployments. EKS is warranted when: needing Kubernetes-specific ecosystem (Helm, Argo CD, Istio, Keda), multi-cloud portability, or advanced scheduling (gang scheduling, topology spread, GPU workloads).

Feature	ECS Fargate	EKS Fargate	EKS EC2
Learning curve	Low (AWS-native API)	High (Kubernetes + AWS)	Very high (K8s + node management)
Control plane cost	Free	\$0.10/hr = \$73/month	\$0.10/hr = \$73/month
Container scheduling	ECS manages placement	Kubernetes scheduler	Kubernetes scheduler + node groups
Autoscaling	Application Auto Scaling + Target Tracking	KEDA, HPA, VPA, Cluster Autoscaler	KEDA, HPA, VPA, Cluster Autoscaler, Karpenter
Service mesh	App Mesh or manual	Istio, Linkerd, App Mesh	Istio, Linkerd — full feature set
Multi-cloud	AWS only	K8s — portable with tooling changes	K8s — portable
Blue/Green deploy	CodeDeploy (built-in)	Argo Rollouts or Flux (additional tooling)	Argo Rollouts or Flux
ShopFlow fit	CURRENT — simple, AWS-native	Consider at 50+ services or K8s ecosystem	Not recommended — Fargate eliminates node man

■ Migrate from ECS to EKS when: (1) your team already has strong Kubernetes expertise, (2) you need advanced traffic management (Istio circuit breaking, mTLS between all services), (3) you want to use KEDA for event-driven autoscaling from 20+ event sources, or (4) regulatory requirement for multi-cloud portability. ECS to EKS migration is non-trivial — task definitions → Helm charts + Kubernetes Deployments; ALB integration → AWS Load Balancer Controller. Plan for 3-6 months migration effort per 10 services.

11-extra ECS Capacity Provider Strategy & Cluster Autoscaling

Capacity Provider Strategy determines how ECS distributes tasks across Fargate and Fargate Spot. The base value guarantees a minimum number of on-demand tasks; weight controls the proportion of additional tasks allocated to each provider. This gives fine-grained control over cost vs availability trade-offs.

Capacity Provider Strategy — Mixed Fargate + Spot

```
// ECS Service: 3 guaranteed on-demand + burst on Spot
const productService = new ecs.FargateService(this,"ProductSvc",{
  ...
  capacityProviderStrategies: [
    {
      capacityProvider: "FARGATE",
      base: 3, // always run at least 3 on-demand tasks
      weight: 1, // 1/4 of tasks beyond base go on-demand
    },
    {
      capacityProvider: "FARGATE_SPOT",
      base: 0,
      weight: 3, // 3/4 of burst tasks go to Spot (70% cheaper)
    },
  ],
});

// Result at Black Friday peak (15 tasks):
// 3 base = on-demand (guaranteed stable)
// 12 additional: 3 on-demand (weight=1) + 9 Spot (weight=3)
// On-demand: 6 tasks × $0.04048/vCPU-hr = $0.243/hr
// Spot: 9 tasks × $0.01214/vCPU-hr = $0.109/hr
// vs all on-demand: 15 × $0.04048 = $0.607/hr
// Saving: ($0.607 - $0.352)/hr = $0.255/hr = $183.60/BF weekend (72hr)
```

Scenario	All On-Demand	3 Base + Spot Burst (3:1 weight)	Saving
Normal (3 tasks)	\$0.121/hr	\$0.121/hr (all base = on-demand)	\$0
Moderate (6 tasks)	\$0.243/hr	\$0.182/hr (3 on-demand + 2.25 Spot avg)	\$61/month
Black Friday (15 tasks)	\$0.607/hr	\$0.352/hr (6 on-demand + 9 Spot)	\$183/BF weekend
Spot interruption (2 tasks lost)	low	ECS re-launches on on-demand automatically (capacity provider fallback)	Site provider fallback below 3 base on-demand

■ When Fargate Spot capacity is unavailable (rare, but possible at high-demand AZs), ECS automatically falls back to on-demand Fargate for the interrupted tasks. The base value of 3 on-demand guarantees your minimum capacity even if all Spot is interrupted simultaneously. Never set base=0 with all Spot — a mass Spot interruption during Black Friday could take your service to 0 tasks.

11-extra2 ECS Task Placement & Spread Strategies

ECS task placement strategies control how tasks are distributed across AZs and hosts. For Fargate, placement strategies ensure tasks spread across availability zones — preventing all tasks from landing in one AZ. Placement constraints enforce compliance requirements (e.g., PCI tasks only on dedicated hosts).

Placement Strategy	Behavior	ShopFlow Use	CDK Config
spread (AZ)	Distribute tasks evenly across AZs	All ECS services — ensures no single-AZ concentration	<code>placementStrategies:[{type:SPREAD,field:attribute:ec2:availabilityZone}]</code>
spread (instanceId)	Distribute across EC2 instances (EC2 only)	None (type) Fargate manages host placement	None for Fargate
binpack (CPU)	Pack tasks on fewest hosts (EC2 only)	Not used — maximize host utilization vs. A/B	None for Fargate
random	Random placement	Dev/test only — no predictability	Not used in production

AZ Spread Strategy CDK — Fargate Service

```
const productService = new ecs.FargateService(this,"ProductSvc",{
```

```

cluster,
taskDefinition: productTaskDef,
desiredCount: 3,
// Spread across AZs — ensures 1 task per AZ minimum
placementStrategies: [
ecs.PlacementStrategy.spreadAcrossInstances(),
],
// Ensure at least 1 AZ always has a running task
vpcSubnets: {
subnets: [
appSubnet1a, // 10.0.3.0/24 in us-east-1a
appSubnet1b, // 10.0.4.0/24 in us-east-1b
appSubnet1c, // 10.0.5.0/24 in us-east-1c
],
},
});
// With desiredCount=3: ECS places 1 task in each AZ (spread)
// During AZ failure (us-east-1a down): 2 remaining tasks serve 100% traffic
// Auto Scaling triggers scale-out: 2 → 4 tasks in remaining 2 AZs

```

Multi-AZ ALB Target Registration

```

// ALB Target Group: register all tasks across all AZs
// ALB distributes requests across all healthy targets
// AZ failover: ALB detects unhealthy targets in 1a; stops routing to 1a
// RT0: ~15 seconds (ALB health check interval: 15s × unhealthy threshold: 2)
// Cross-zone load balancing (enabled by default on ALB):
// Each ALB node distributes requests to ALL targets in ALL AZs
// Prevents AZ imbalance even when target counts differ between AZs
const targetGroup = new elbv2.ApplicationTargetGroup(this, "ProductTG", {
vpc,
port: 8080,
protocol: elbv2.ApplicationProtocol.HTTP,
targetType: elbv2.TargetType.IP, // Fargate uses IP targets (not instance)
healthCheck: {
path: "/actuator/health",
interval: Duration.seconds(15),
healthyThresholdCount: 3,
unhealthyThresholdCount: 2,
},
deregistrationDelay: Duration.seconds(30),
});

```

■ Fargate tasks use IP-based target registration (not instance-based). Each task gets a private IP from the VPC subnet CIDR. ALB registers the task IP directly — no need to manage instance IDs or port mapping. This enables precise health checking at the container level. If a container crashes but the EC2 host is fine, the ALB detects the health check failure at the container IP and stops routing to that specific task.

11-BP

Best Practices & Anti-Patterns

- ✓ Use ARM64 (Graviton3) Fargate tasks — 20% cost reduction, same pricing tier, no code changes for Java
- ✓ Set `-XX:MaxRAMPercentage=75.0` — prevents JVM under-utilizing container memory (25% default is too low)
- ✓ Enable deployment circuit breaker with rollback — auto-revert bad deployments before they bring all tasks down
- ✓ Set deregistration delay to 30s for REST APIs — rolling deploys complete in minutes not 15+ minutes
- ✓ Use Fargate Spot for batch workloads — 70% discount; implement SIGTERM handler for graceful drain
- ✓ Enable ECS Exec for incident response — interactive shell without SSH or bastion host
- ✓ Use Secrets Manager for all sensitive config — ECS injects at task start; never in Dockerfile ENV
- ✓ Use Cloud Map DNS for service-to-service calls — DNS TTL 10s for fast failover

- ✓ Set healthy threshold=3 and interval=15s for health checks — prevents traffic to partially-started Spring Boot
- ✓ Use Container Insights for CPU/memory metrics — right-size tasks based on actual utilization (>40% of spec)
- ✓ Use immutable ECR image tags (imageTagMutability: IMMUTABLE) — guarantees deployed image = tested image
- ✓ Apply Compute Savings Plans for committed baseline — 36% off 1-year; covers Fargate + Lambda + EC2
- ✓ Use 3-tier subnet design: public (ALB), app (ECS), data (Aurora/Redis) — minimize blast radius
- ✓ Set scaleInCooldown=5min, scaleOutCooldown=60s — aggressive scale-out, conservative scale-in
- ✓ Use Blue/Green (CodeDeploy) for financial services — zero-downtime for order-svc and cart-svc
- Never use default Fargate CPU/memory without profiling — Java needs MaxRAMPercentage to use full container memory
- Do not disable health checks — ECS will route traffic to crashed tasks; always use /actuator/health
- Avoid Fargate Spot for stateful services — interruption loses in-memory state; use only for stateless batch
- Do not share ECR repositories across environments — separate ECR repo per environment (dev/staging/prod)
- Avoid large deregistration delay (default 300s) for REST APIs — rolling deploys block for 5min per task
- Never put credentials in ECS environment variables plaintext — use Secrets Manager injection instead
- Do not use public subnets for ECS tasks — app-tier private subnets only; outbound via NAT Gateway

11-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
Fargate pricing	\$0.04048/vCPU-hr + \$0.004445/GB-hr. ARM64 (Graviton): 20% off. Spot: up to 70% off.
ECS health check path	/actuator/health (Spring Boot Actuator). Returns 200=healthy, 503=unhealthy. Check includes DB + Redis
Deployment circuit breaker	Rolls back automatically if >50% of new tasks fail health checks in deployment window. Enable rollback
Deregistration delay	Time ALB waits before closing connections to deregistered task. Default: 300s. Recommended for REST
Fargate Spot interrupt	2-minute SIGTERM warning before interruption. Implement ContextClosedEvent listener; re-queue in-flight
ECS Exec transport	SSM Session Manager — no inbound port 22 needed. Requires SSM Agent in container + ssm:StartSess
Cloud Map DNS TTL	Set to 10s — unhealthy task removed from DNS within 10s of health check failure.
ECR image tag immutable	imageTagMutability: IMMUTABLE prevents tag overwrite. Force version bump per deploy. Guarantees de
Compute Savings Plans	Cover Fargate + Lambda + EC2 with one plan. 1-year no-upfront: 36% off. 3-year: 52% off. Applies autom
ECS vs EKS decision	ECS: simpler, AWS-native, lower ops overhead. EKS: Kubernetes ecosystem, multi-cloud, advanced sche

Top Interview Questions

Question	Answer
How does ECS rolling deploy work?	ECS launches new tasks (up to maximumPercent=200, so 2× desired count). New tasks pass health check
Fargate Spot interruption handling	ECS sends SIGTERM 2 minutes before interruption. Spring Boot listens via ContextClosedEvent. Handler
ECS secrets — how are they injected?	EC2 task definition secrets field references Secrets Manager ARN. ECS execution role has GetSecretVal
When to use Blue/Green vs Rolling update?	Rolling update: simpler, no 2× task cost, sufficient for most services. Slight traffic shift during deploy (old a
ECS Auto Scaling — how does it work?	Application Auto Scaling controls ECS desired count. Two policies work simultaneously: Target Tracking (